

Understanding compositional generalisation in neural nets using datasets generated with GF

Anssi Moisio
Aalto University, Finland

Grammatical Framework Summer School
28th August 2025, Gothenburg



Aalto University

Outline of the presentation

- Background: previous work on evaluating compositional generalisation (CG) in neural networks
 - **The research question:** What properties in the training data help neural networks in CG?
 - **A hypothesis:** Structural variation
- Extending compositional generalisation benchmarks with more structural variation using GF

**Background: benchmarking compositional generalisation in
neural network sequence-to-sequence models**

SCAN benchmark for evaluating compositional generalisation (Lake and Baroni, 2018)

e.g.:

INPUT: "look twice after run opposite left"

OUTPUT: "I_TURN_LEFT I_TURN_LEFT I_RUN I_LOOK I_LOOK"

```
abstract Scan = {  
  flags startcat = Utt ;  
  
  cat Utt ; ConjImp ; Imp ; VP ; Adv ; Verb ; VD ; V ;  
  
  data  
    UseConjImp : ConjImp -> Utt ;  
    UseImp     : Imp     -> Utt ;  
  
    CoordImp   : Imp -> Imp -> ConjImp ; -- walk and jump  
    CoordImpInv : Imp -> Imp -> ConjImp ; -- walk after jump  
  
    ImpVP      : VP -> Imp ;  
    Twice      : VP -> Imp ; -- walk twice  
    Thrice     : VP -> Imp ; -- walk thrice  
  
    UseV       : V       -> VP ;  
    DirVP      : Verb -> Adv -> VP ; -- walk left  
    OppositeVP : Verb -> Adv -> VP ; -- walk opposite left  
    AroundVP   : Verb -> Adv -> VP ; -- walk around left  
  
    VVerb      : V       -> Verb ;  
    VDVerb     : VD      -> Verb ;  
  
    left_Adv, right_Adv : Adv ;  
    turn_VD            : VD ;  
    walk_V, run_V, jump_V, look_V : V ;  
}
```

```
concrete ScanInput of Scan = {  
  
  lincat Utt, ConjImp, Imp, VP, Adv, Verb, VD, V = {s : Str};  
  
  lin  
    UseConjImp imp = imp ;  
    UseImp      imp = imp ;  
  
    CoordImp   imp1 imp2 = {s = imp1.s ++ "and" ++ imp2.s} ;  
    CoordImpInv imp1 imp2 = {s = imp1.s ++ "after" ++ imp2.s} ;  
  
    ImpVP      vp = vp ;  
    Twice      vp = {s = vp.s ++ "twice"} ;  
    Thrice     vp = {s = vp.s ++ "thrice"} ;  
  
    UseV       v = v ;  
    DirVP      v adv = {s = v.s ++ adv.s} ;  
    OppositeVP v adv = {s = v.s ++ "opposite" ++ adv.s} ;  
    AroundVP   v adv = {s = v.s ++ "around" ++ adv.s} ;  
  
    VVerb      v = v ;  
    VDVerb     v = v ;  
  
    left_Adv = {s = "left"} ;  
    right_Adv = {s = "right"} ;  
    turn_VD = {s = "turn"} ;  
    walk_V = {s = "walk"} ;  
    run_V = {s = "run"} ;  
    jump_V = {s = "jump"} ;  
    look_V = {s = "look"} ;  
}
```

```
concrete ScanOutput of Scan = {  
  
  lincat Utt, ConjImp, Imp, VP, Adv, Verb, VD, V = {s : Str};  
  
  lin  
    UseConjImp imp = imp ;  
    UseImp      imp = imp ;  
  
    CoordImp   imp1 imp2 = {s = imp1.s ++ imp2.s} ;  
    CoordImpInv imp1 imp2 = {s = imp2.s ++ imp1.s} ;  
  
    ImpVP      vp = vp ;  
    Twice      vp = {s = vp.s ++ vp.s} ;  
    Thrice     vp = {s = vp.s ++ vp.s ++ vp.s ++ vp.s} ;  
  
    UseV       v = v ;  
    DirVP      v adv = {s = adv.s ++ v.s} ;  
    OppositeVP v adv = {s = adv.s ++ adv.s ++ v.s} ;  
    AroundVP   v adv = {s = adv.s ++ v.s ++  
                        adv.s ++ v.s ++  
                        adv.s ++ v.s} ;  
  
    VVerb      v = v ;  
    VDVerb     _ = {s = []} ;  
  
    left_Adv = {s = "I_TURN_LEFT"} ;  
    right_Adv = {s = "I_TURN_RIGHT"} ;  
    turn_VD = {s = ""} ;  
    walk_V = {s = "I_WALK"} ;  
    run_V = {s = "I_RUN"} ;  
    jump_V = {s = "I_JUMP"} ;  
    look_V = {s = "I_LOOK"} ;  
}
```

SCAN benchmark for evaluating compositional generalisation (Lake and Baroni, 2018)

e.g.:

INPUT: “look twice after run opposite left”

OUTPUT: “I_TURN_LEFT I_TURN_LEFT I_RUN I_LOOK I_LOOK”

```
abstract Scan = {
  flags startcat = Utt ;

  cat Utt ; ConjImp ; Imp ; VP ; Adv ; Verb ; VD ; V ;

  data
    UseConjImp : ConjImp -> Utt ;
    UseImp : Imp -> Utt ;

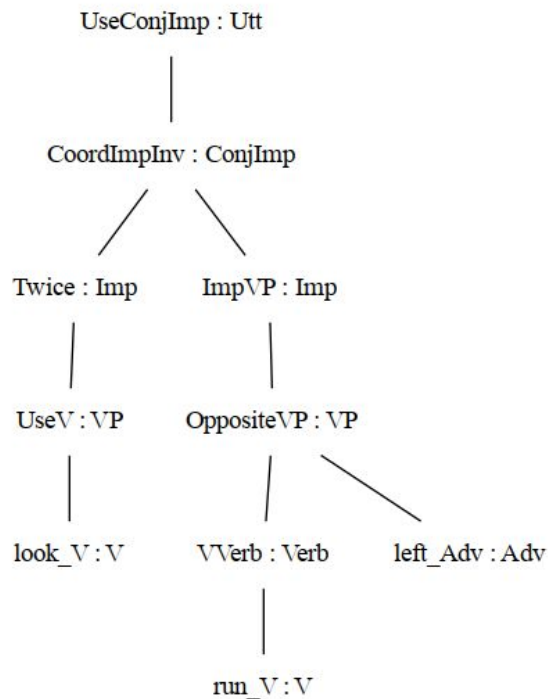
    CoordImp : Imp -> Imp -> ConjImp ; -- walk and jump
    CoordImpInv : Imp -> Imp -> ConjImp ; -- walk after jump

    ImpVP : VP -> Imp ;
    Twice : VP -> Imp ; -- walk twice
    Thrice : VP -> Imp ; -- walk thrice

    UseV : V -> VP ;
    DirVP : Verb -> Adv -> VP ; -- walk left
    OppositeVP : Verb -> Adv -> VP ; -- walk opposite left
    AroundVP : Verb -> Adv -> VP ; -- walk around left

    VVerb : V -> Verb ;
    VDVerb : VD -> Verb ;

    left_Adv, right_Adv : Adv ;
    turn_VD : VD ;
    walk_V, run_V, jump_V, look_V : V ;
}
```



SCAN benchmark for evaluating compositional generalisation (Lake and Baroni, 2018)

e.g.:

INPUT: “look twice after run opposite left”

OUTPUT: “I_TURN_LEFT I_TURN_LEFT I_RUN I_LOOK I_LOOK”

```
abstract Scan = {
  flags startcat = Utt ;

  cat Utt ; ConjImp ; Imp ; VP ; Adv ; Verb ; VD ; V ;

  data
    UseConjImp : ConjImp -> Utt ;
    UseImp      : Imp      -> Utt ;

    CoordImp    : Imp -> Imp -> ConjImp ; -- walk and jump
    CoordImpInv : Imp -> Imp -> ConjImp ; -- walk after jump

    ImpVP      : VP -> Imp ;
    Twice      : VP -> Imp ; -- walk twice
    Thrice     : VP -> Imp ; -- walk thrice

    UseV      : V      -> VP ;
    DirVP     : Verb -> Adv -> VP ; -- walk left
    OppositeVP : Verb -> Adv -> VP ; -- walk opposite left
    AroundVP  : Verb -> Adv -> VP ; -- walk around left

    VVerb      : V      -> Verb ;
    VDVerb     : VD     -> Verb ;

    left_Adv, right_Adv      : Adv ;
    turn_VD                  : VD ;
    walk_V, run_V, jump_V, look_V : V ;
}
```

```
abstract ScanAddJump = Scan
- [jump_V]
**
{
  flags startcat = Utt ;

  cat VJ ;

  data
    UseVJ      : VJ -> Utt ;
    jump_VJ    : VJ ;
}
```

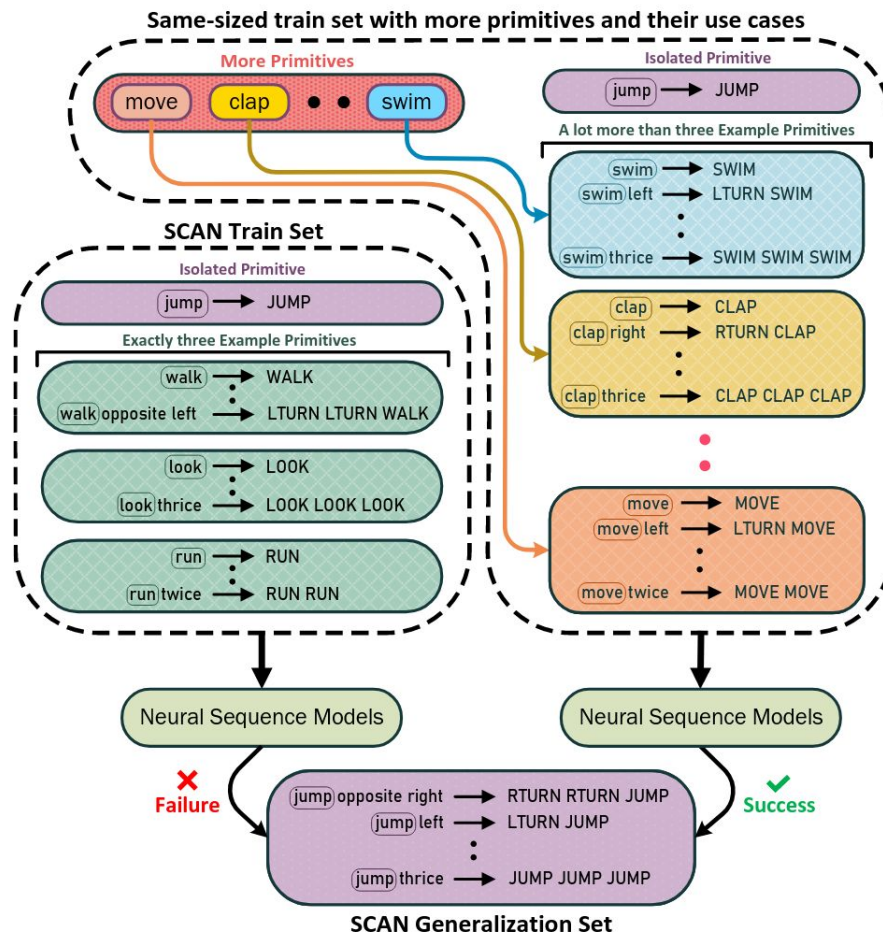
Generalisation test set:

- systematically remove specific structures from the training set
- test whether sequence-to-sequence models can still translate those structures

Result: they can't

→ conclusion: the tested neural networks are not able to generalise compositionally

SCAN revisited (Patel et al., 2022)



SCAN revisited (Patel et al., 2022)

e.g.:

INPUT: “look twice after run opposite left”

OUTPUT: “I_TURN_LEFT I_TURN_LEFT I_RUN I_LOOK I_LOOK”

```
abstract Scan = {
  flags startcat = Utt ;

  cat Utt ; ConjImp ; Imp ; VP ; Adv ; Verb ; VD ; V ;

  data
    UseConjImp : ConjImp -> Utt ;
    UseImp     : Imp     -> Utt ;

    CoordImp   : Imp -> Imp -> ConjImp ; -- walk and jump
    CoordImpInv : Imp -> Imp -> ConjImp ; -- walk after jump

    ImpVP      : VP -> Imp ;
    Twice      : VP -> Imp ; -- walk twice
    Thrice     : VP -> Imp ; -- walk thrice

    UseV       : V       -> VP ;
    DirVP      : Verb -> Adv -> VP ; -- walk left
    OppositeVP : Verb -> Adv -> VP ; -- walk opposite left
    AroundVP   : Verb -> Adv -> VP ; -- walk around left

    VVerb      : V       -> Verb ;
    VDVerb     : VD      -> Verb ;

    left_Adv, right_Adv : Adv ;
    turn_VD             : VD ;
    walk_V, run_V, jump_V, look_V : V ;
}
```

```
abstract ScanAddJump = Scan
- [jump_V]
**
{
  flags startcat = Utt ;

  cat VJ ;

  data
    UseVJ       : VJ -> Utt ;
    jump_VJ     : VJ ;
}
```

Generalisation test set:

- systematically remove specific structures from the training set
- test whether sequence-to-sequence models can still translate those structures

Result: they can't

→ conclusion: the tested neural networks are not able to generalise compositionally

BUT if you add 50 new verbs here, they are. (Patel et al. 2022)

→ conclusion: ??

Properties of training data that enable compositional generalisation

- Patel et al. (2022): a large enough number of words in a word class helps learning the grammatical structures the words of that class can appear in
 - enables applying the grammatical structure to new words
 - “lexical generalisation”
 - but then there’s also “structural generalisation”

COGS: A Compositional Generalization Challenge Based on Semantic Interpretation (Kim and Linzen, 2020)

A semantic parsing task, e.g.: *A hedgehog ate the cake* $\rightarrow$ *cake(x_4) ; hedgehog(x_1) AND eat.agent(x_2, x_1) AND eat.theme(x_2, x_4)

With different generalisation cases:

Case	Training	Generalization
S.3.2. Novel Combination Modified Phrases and Grammatical Roles		
Object modification $\rightarrow$ Subject modification	Noah ate the cake on the plate.	The cake on the table burned.
S.3.3. Deeper Recursion		
Depth generalization: Sentential complements	Emma said that Noah knew that the cat danced.	Emma said that Noah knew that Lucas saw that the cat danced.
Depth generalization: PP modifiers	Ava saw the ball in the bottle on the table.	Ava saw the ball in the bottle on the table on the floor.
S.3.4. Verb Argument Structure Alternation		
Active $\rightarrow$ Passive	The crocodile blessed William.	A muffin was blessed .
Passive $\rightarrow$ Active	The book was squeezed .	The girl squeezed the strawberry.
Object-omitted transitive $\rightarrow$ Transitive	Emily baked .	The giraffe baked a cake .
Unaccusative $\rightarrow$ Transitive	The glass shattered .	Liam shattered the jigsaw.
Double object dative $\rightarrow$ PP dative	The girl teleported Liam the cookie.	Benjamin teleported the cake to Isabella.
PP dative $\rightarrow$ Double Object Dative	Jane shipped the cake to John.	Jane shipped John the cake.

Lexical and structural generalisation

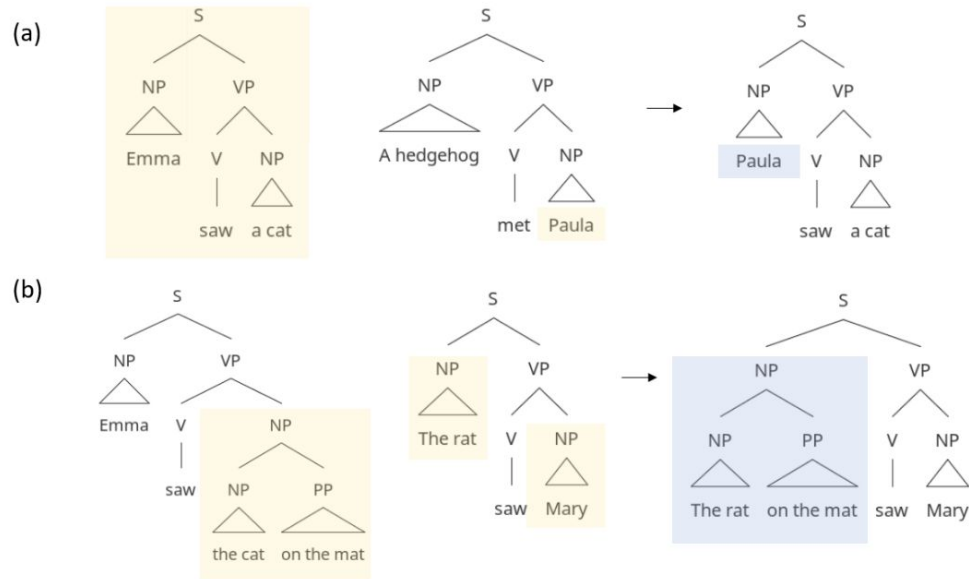


Figure 3: (a) Lexical generalization: a novel combination of a familiar primitive and a familiar structure. (b) Structural generalization: a novel combination of two familiar structures.

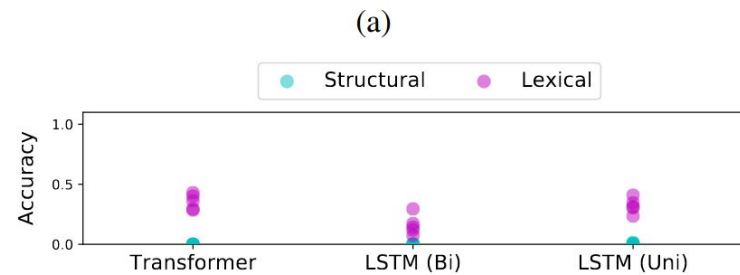
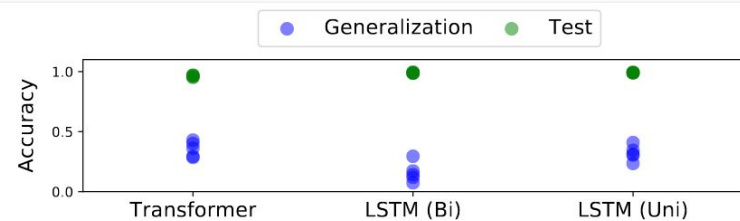


Figure 2: (a) Accuracy on COGS. An output sequence is considered correct only if it exactly matches the gold sequence. Each dot represents a model trained with a different random seed. (b) Accuracy by generalization type (lexical or structural).

Properties of training data that enable compositional generalisation

- Patel et al. (2022): a large enough number of words in a word class enables learning the grammatical structures the words in that class can appear in
 - enables applying the grammatical structure to new words
 - “lexical generalisation”
- hypothesis:
 - analogously for “structural generalisation”: a large enough number of e.g. different types of noun phrases in the subject and object positions enable learning that any noun phrase can be a subject or object
 - enables using new noun phrases as the subject or object

Extending the COGS dataset using GF

Extending the COGS dataset

TRAINING

Noah ate **a cake on the plate**

Noah's cake smelled nice

A big red house froze

The third pizza burned

There was **a cake on the plate**

...

GENERALISATION

A cake on the table burned

Extending the COGS dataset

A hedgehog ate the cake $\rightarrow$ *cake(x_4) ; hedgehog(x_1) AND eat.agent(x_2, x_1) AND eat.theme(x_2, x_4)

Neo-Davidsonian logical semantics

```
1 abstract ExampleSemantics = MiniGrammar ** {
2
3   flags startcat = Prop ;
4
5   cat Prop ; Ind ; Event ;
6
7   fun iS : S -> Prop ;
8   def iS (UseCl t p (PredVP np vp)) = ExistE (iTense t (iNP np (iPol p (iVP vp)))) ;
9
10  fun iTense : Tense -> (Event -> Prop) -> Event -> Prop ;
11  def iTense t P = \e -> And (P e) (Time t e) ;
12
13  fun iNP : NP -> (Ind -> Event -> Prop) -> Event -> Prop ;
14  def iNP (DetCN det cn) vp = iDet det (iCN cn) vp ;
15
16  -- Polarity modifies the interpreted verb phrase
17  fun iPol : Pol -> (Ind -> Event -> Prop) -> (Ind -> Event -> Prop) ;
18  def
19    iPol PPos vp = vp ;
20    iPol PNeg vp = \x,e -> Not (vp x e) ;
21
22  fun iVP : VP -> Ind -> Event -> Prop ;
23  def
24    iVP (UseV v) i = iV v i ;
25    -- The object NP takes the transitive verb as its scope.
26    -- i is the subject and y is the direct object variable
27    iVP (ComplV2 v np) i = iNP np (\y -> iV2 v i y) ;
28
29
```

```
29
30 -- A noun and a verb phrase combine into a proposition of event
31 fun iDet : Det -> (Ind -> Prop) -> (Ind -> Event -> Prop) -> Event -> Prop ;
32 def
33   iDet a_Det n vp = \e -> Exist (\x -> And (n x) (vp x e)) ;
34   iDet every_Det n vp = \e -> All (\x -> If (n x) (vp x e)) ;
35
36 -- A verb is a proposition about 1-2 individual(s) and an event
37 fun
38   iV : V -> Ind -> Event -> Prop ;
39   iV2 : V2 -> Ind -> Ind -> Event -> Prop ;
40 def
41   iV v subj e = Agent (VVerb v) subj e ;
42   iV2 v subj obj e = And (Agent (V2Verb v) subj e) (Theme (V2Verb v) obj e) ;
43
44 -- A common noun is a proposition about an individual
45 fun iCN : CN -> Ind -> Prop ;
46 -- interpretation stops at nouns
47
48 fun
49   Agent : Verb -> Ind -> Event -> Prop ;
50   Theme : Verb -> Ind -> Event -> Prop ;
51   Time : Tense -> Event -> Prop ;
52
53   And, Or, If : Prop -> Prop -> Prop ;
54   Not : Prop -> Prop ;
55   All, Exist : (Ind -> Prop) -> Prop ;
56   ExistE : (Event -> Prop) -> Prop ;
57
58 }
59
```


Neo-Davidsonian logical semantics

"a boy did not like that the man has not painted a house"

abstract syntax tree

```
ExistE (\v0 -> And
  (Exist (\v1 -> And
    (iN boy_N v1)
    (Not (And
      (Agent like_VS v1 v0)
      (ExistE (\v2 -> And
        (Ccomp like_VS v2 v0)
        (And
          (Exist (\v3 -> And
            (Unique (\v4 -> iN man_N v4) v3)
            (Not (Exist (\v4 -> And
              (iN house_N v4)
              (And
                (Agent paint_V2 v3 v2)
                (Theme paint_V2 v4 v2)
              )
            )))
          ))
        (Time v2 Perf)
      )
    ))
  ))
  (Time v0 Imp)
)
```

linearisation

```
* man ( v3 ) ;
boy ( v1 ) AND
NOT(
  like . agent ( v0 , v1 ) AND
  like . ccomp ( v0 , v2 ) AND
  NOT(
    house ( v4 ) AND
    paint . agent ( v2 , v3 ) AND
    paint . theme ( v2 , v4 )
  ) AND
  TIME ( v2 , perf )
) AND
TIME ( v0 , imp )
```

Current status, open questions, and next steps

Done:

- a grammar and a logical semantics for a subset of COGS
- a grammar and a logical semantics for some structures that are not in the original COGS dataset
 - tense
 - polarity

Open questions:

- the exact distribution of the generated dataset
 - selectional restrictions

The steps to complete the study:

- generate a version of the COGS datasets with more structural variation
- train transformer sequence-to-sequence models on the data
 - → hypothesis verified?

References

- Najoung Kim and Tal Linzen. 2020. COGS: A compositional generalization challenge based on semantic interpretation. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pages 9087–9105, Online. Association for Computational Linguistics
- Brenden Lake and Marco Baroni. 2018. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In International Conference on Machine Learning, pages 2873–2882. PMLR.
- Arkil Patel, Satwik Bhattamishra, Phil Blunsom, and Navin Goyal. 2022. Revisiting the Compositional Generalization Abilities of Neural Sequence Models. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), pages 424–434, Dublin, Ireland. Association for Computational Linguistics.

Thank you